

Heuristic_Search_Tool

```
import copy

# Notice: No more hardcoded import here! The tool is now 100%
independent.

def check_constraints(specs, constraints):
    """
    Helper function to check if the current specs pass all original
    constraints.
    constraints format: {'PM': ('>=', 60.0), 'Pw': ('<=', 300e-6), ...}
    """
    for spec_name, (operator, target_val) in constraints.items():
        if spec_name not in specs:
            continue # Skip if spec isn't in the output

        current_val = specs[spec_name]
        if operator == '>=' and current_val < target_val:
            return False
        elif operator == '<=' and current_val > target_val:
            return False
    return True

def run_adaptive_search(
    simulator_func, param_to_change, direction, target_spec,
    target_value, target_operator,
    current_x, current_specs, constraints, static_constants, budget
):
    """
    Executes the Adaptive Step-Size Heuristic Search.

    Returns a dictionary:
    {
        'status': str,
        'sims_used': int,
        'safe_trials_log': list of dicts (contains x_params and specs)
    }

```

```

}
"""

print(f"\n--- Starting Adaptive Search on {param_to_change}
({direction}) ---")

# Initialization
x_safe = copy.deepcopy(current_x)
spec_safe = copy.deepcopy(current_specs)

base_step_pct = 0.05
success_counter = 0
sims_used = 0
safe_trials = []

# Minimum improvement threshold (1%)
min_improvement_ratio = 1.01 if target_operator == '>=' else 0.99

while sims_used < budget:
    # 1. Calculate multiplier based on direction and step percentage
    if direction == "increase":
        multiplier = 1.0 + base_step_pct
    else:
        multiplier = 1.0 - base_step_pct

    # 2. Modify parameter
    x_test = copy.deepcopy(x_safe)
    x_test[param_to_change] *= multiplier

    print(f"Trial {sims_used + 1}: Testing {param_to_change} =
{x_test[param_to_change]:.4e} (Step: {base_step_pct*100:.1f}%)")

    # 3. Simulate
    try:
        # Combine tuning params with static params for the simulator
wrapper
        sim_inputs = {**x_test, **static_constants}
        # THE MAGIC HAPPENS HERE: We call whatever function the Agent
passed to us!
        test_specs = simulator_func(**sim_inputs)

```

```

        sims_used += 1

    except Exception as e:
        print(f"Simulation failed (Likely pushed into invalid
geometry): {e}")
        print("Result: Simulator crash. Treating as Overshoot.
Backtracking...")
        base_step_pct /= 2.0 # Halve the step size
        success_counter = 0 # Reset acceleration

        # If step becomes microscopically small, break
        if base_step_pct < 0.005:
            print(f"Step size too small ({base_step_pct*100:.2f}%).
Discarding {param_to_change}.")
            sims_used += 1
            break

        sims_used += 1
        continue # Try again with the halved step size!

# 4. Check if Target Spec Improved by at least 1%
old_val = spec_safe[target_spec]
new_val = test_specs[target_spec]

improved_enough = False
if target_operator == '>=' and new_val >= (old_val * 1.01):
    improved_enough = True
elif target_operator == '<=' and new_val <= (old_val * 0.99):
    improved_enough = True

target_hit = False
if target_operator == '>=' and new_val >= target_value:
    target_hit = True
elif target_operator == '<=' and new_val <= target_value:
    target_hit = True

if not improved_enough and not target_hit:
    print(f"Result: Spec plateaued or improvement < 1%.
Discarding {param_to_change}.")

```

```

        break # Exit the loop, force agent to pick contingency
parameter

# 5. Check if other constraints passed
constraints_passed = check_constraints(test_specs, constraints)

if constraints_passed:
    # SUCCESS PATH
    print("Result: Success! Target improved and constraints
passed.")
    x_safe = copy.deepcopy(x_test)
    spec_safe = copy.deepcopy(test_specs)

    # Log the safe trial
    safe_trials.append({
        'params': copy.deepcopy(x_safe),
        'specs': copy.deepcopy(spec_safe)
    })

    if target_hit:
        print(f"*** TARGET ACHIEVED for {target_spec} ***")
        return {
            'status': 'Target Reached',
            'sims_used': sims_used,
            'safe_trials_log': safe_trials
        }

    success_counter += 1
    if success_counter >= 2:
        base_step_pct *= 2.0 # Acceleration!
        print(f"Gradient stable. Accelerating step size to
{base_step_pct*100:.1f}%.")

    else:
        # OVERSHOOT PATH (Backtracking)
        print("Result: Overshoot. Constraints failed.
Backtracking...")
        base_step_pct /= 2.0 # Halve the step size
        success_counter = 0 # Reset acceleration

```

```
# If step becomes microscopically small, break
if base_step_pct < 0.005:
    print(f"Step size too small ({base_step_pct*100:.2f}%).
Discarding {param_to_change}.")
    break

# Loop exited (either budget exhausted or variable discarded)
status_msg = "Budget Exhausted" if sims_used >= budget else "Variable
Discarded"

return {
    'status': status_msg,
    'sims_used': sims_used,
    'safe_trials_log': safe_trials
}
```